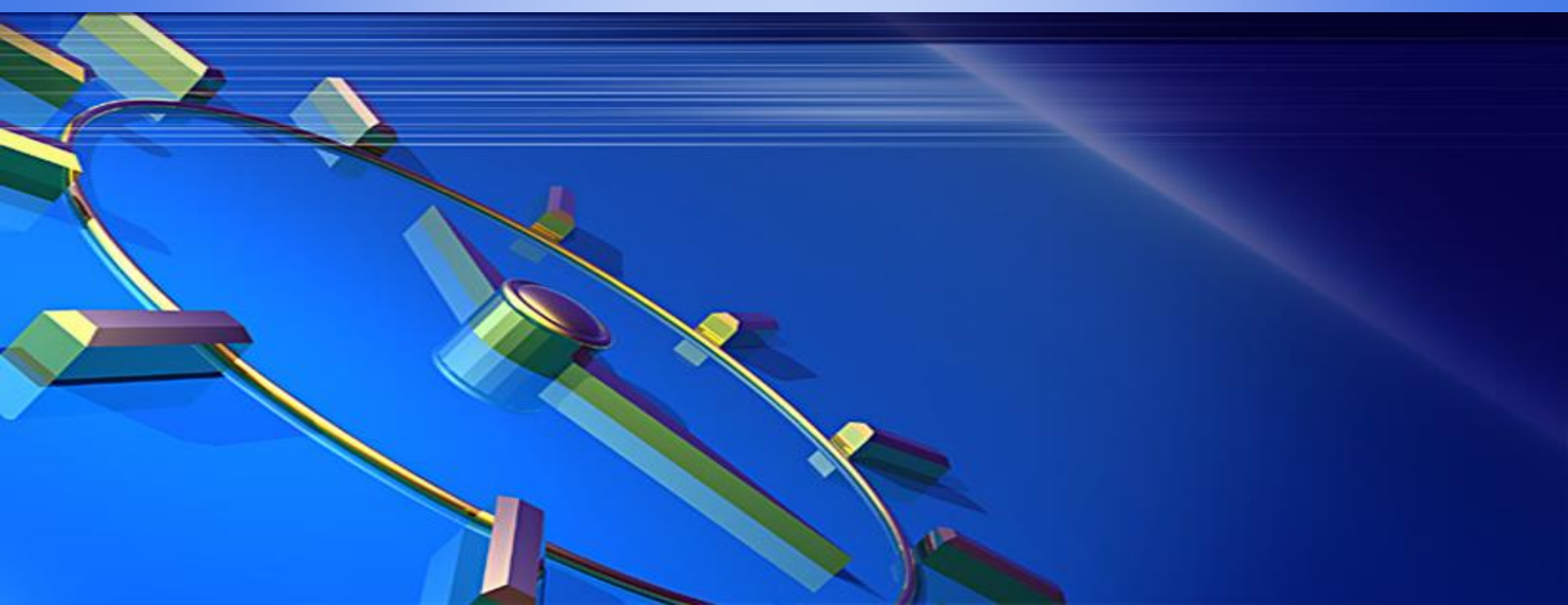




Tema 4. Fiabilidad



Contenido



A diagram showing a table of contents. On the left, a large light blue circle is partially visible. A grey curved line starts from the top left and extends downwards, passing through five colored circular markers (yellow, green, blue, purple, orange). Each marker is followed by a white rounded rectangular box containing text. The text boxes are arranged horizontally to the right of the markers.

Que es fiabilidad

Fiabilidad de Software

Evaluación de la fiabilidad

Tipos de Fallos

Prevencion y Tolerancia a Fallos



¿Qué es la fiabilidad?

Fiabilidad:

Permanencia de la calidad de los productos (o servicios) a lo largo del tiempo.
Capacidad de desarrollar adecuadamente su labor a lo largo del tiempo.

Definición AFNOR X 06-501

“Fiabilidad es la característica de un dispositivo expresada por la probabilidad de que un dispositivo cumpla una función requerida en las condiciones de utilización y para un período de tiempo determinado”.

Probabilidad: es la relación número de casos favorables número de casos posibles asociada a un tiempo t .

Se denomina $R(t) = P(\text{cumplir una misión}) = P(\text{buen funcionamiento})$
R traducción del inglés *Reliability*



La Fiabilidad intenta garantizar que el producto permanecerá en buenas condiciones durante un periodo razonable de tiempo

Según la definiciones se tiene que pensar muy claramente qué significa

- Funcionamiento satisfactorio
- Tiempo de funcionamiento (Misión)
- Condiciones de funcionamiento

Necesidad de fiabilidad

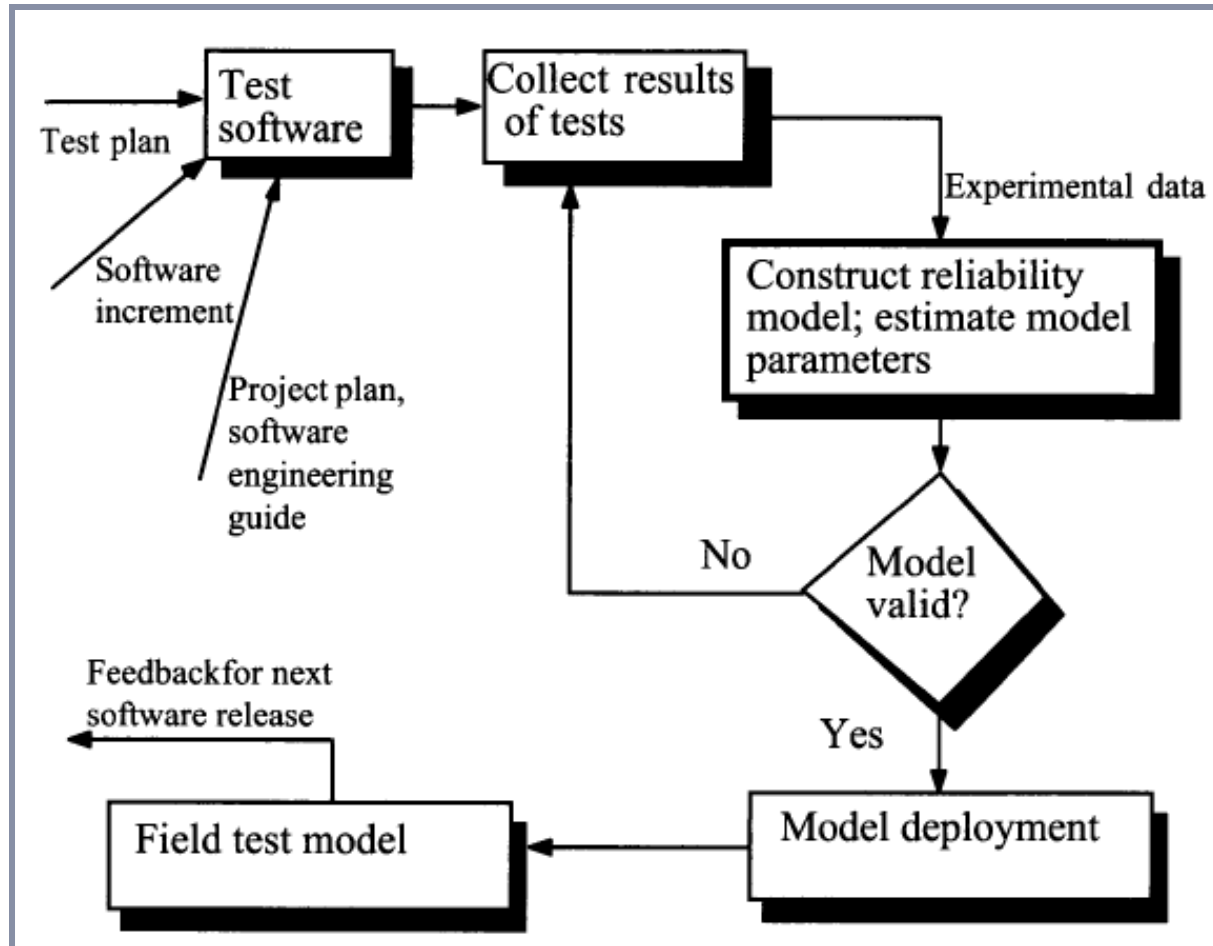
- Desde un punto de vista puramente económico, es deseable una alta fiabilidad para reducir los costos totales del producto.
- El hecho de que en algunos sistemas militares el costo anual de mantenimiento sea diez veces el costo original del mismo, pone de manifiesto esta necesidad. (ciclo vida)
- También hay que considerar el aspecto de seguridad (el fallo de un sistema ABS en un automóvil puede ser catastrófico).
- Existen otro aspectos como retrasos de horarios, incomodidades, insatisfacción del cliente y pérdida de prestigio del fabricante.
- Cada vez son más las empresas y organismos que en sus contrataciones exigen ciertas normas de fiabilidad (MIL HDBK 217 en USA...)



Fiabilidad de software

- ❖ **Fiabilidad, capacidad del software de mantener las prestaciones requeridas del sistema, durante un tiempo establecido y bajo un conjunto de condiciones definidas.**

Evaluacion de Fiabilidad



El Rol de la Fiabilidad en Ingenieria de Software

La confiabilidad es probablemente la más importante de las características inherentes en el concepto de "calidad del software". La confiabilidad del software se refiere a qué tan bien funciona el software para cumplir con los requisitos del cliente.

j.Musa y otros 1990

Error, Fault and Failure



- ❖ **Error**: human action that results in software containing a fault
- ❖ **Fault**: a cause for an internal error (failure)
- ❖ **Failure**: any observable divergence of software behavior in execution from user needs
- ❖ **Failure intensity**: the number of failures per time unit



Fallos de funcionamiento

- **Los fallos de funcionamiento de un sistema pueden tener su origen en**
 - Una especificación inadecuada
 - Errores de diseño del software
 - Averías en el hardware
 - Interferencias transitorias o permanentes en las comunicaciones u
- **Nos centraremos en el estudio de los errores de software**

- 
- **La fiabilidad (reliability) de un sistema es una medida de su conformidad con una especificación autorizada de su comportamiento**
 - **Una avería (failure) es una desviación del comportamiento de un sistema respecto de su especificación**
 - **Las averías se manifiestan en el comportamiento externo del sistema, pero son el resultado de errores (errors) internos**
 - **Las causas mecánicas o algorítmicas de los errores se llaman fallos (faults)**

- **Los fallos pueden ser consecuencia de averías en los componentes del sistema (que son también sistemas)**



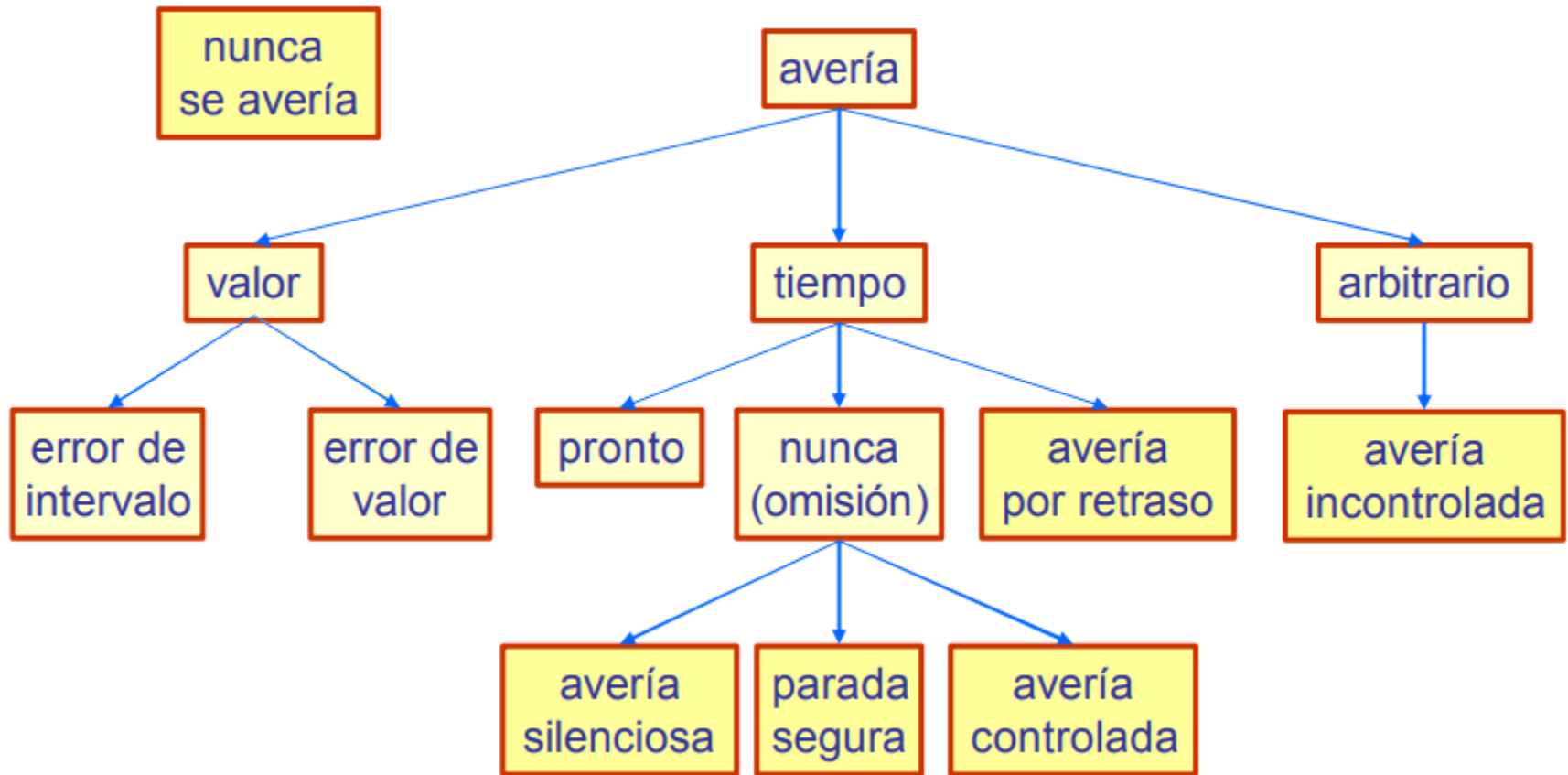


Tipos de fallos

- **Fallos transitorios**
 - desaparecen solos al cabo de un tiempo
 - ejemplo: interferencias en comunicaciones
- **Fallos permanentes**
 - permanecen hasta que se reparan
 - ejemplo: roturas de hardware, errores de diseño de software
- **Fallos intermitentes**
 - fallos transitorios que ocurren de vez en cuando
 - ejemplo: calentamiento de un componente de hardware

Debe impedirse que los fallos de todos estos tipos causen averías

Tipos de avería (failure modes)





Prevención y tolerancia de fallos

- **Hay dos formas de aumentar la fiabilidad de un sistema:**
 - Prevención de fallos
 - Se trata de evitar que se introduzcan fallos en el sistema antes de que entre en funcionamiento
 - Tolerancia de fallos
 - Se trata de conseguir que el sistema continúe funcionando aunque se produzcan fallos
- **En ambos casos el objetivo es desarrollar sistemas con tipos de averías bien definidos**



Prevención de fallos

❖ Se realiza en dos etapas:

- Evitación de fallos
 - Se trata de impedir que se introduzcan fallos durante la construcción del sistema
- Eliminación de fallos
 - Consiste en encontrar y eliminar los fallos que se producen en el sistema una vez construido



Técnicas de evitación de fallos

- **Hardware**

- Utilización de componentes fiables
- Técnicas rigurosas de montaje de subsistemas
- Apantallamiento de hardware

- **Software**

- Especificación de requisitos rigurosa o formal
- Métodos de diseño comprobados
- Lenguajes con abstracción de datos y modularidad
- Utilización de entornos de desarrollo con computador (CASE) adecuados para gestionar los componentes



Técnicas de eliminación de fallos

- **Comprobaciones**

- Revisiones de diseño
- Verificación de programas
- Inspección de código

- **Pruebas (tests)**

- Son necesarias, pero tienen problemas:
 - no pueden ser nunca exhaustivas
 - sólo sirven para mostrar que hay errores, no que no los hay
 - a menudo es imposible reproducir las condiciones reales
 - los errores de especificación no se detectan



Limitaciones de la prevención de fallos

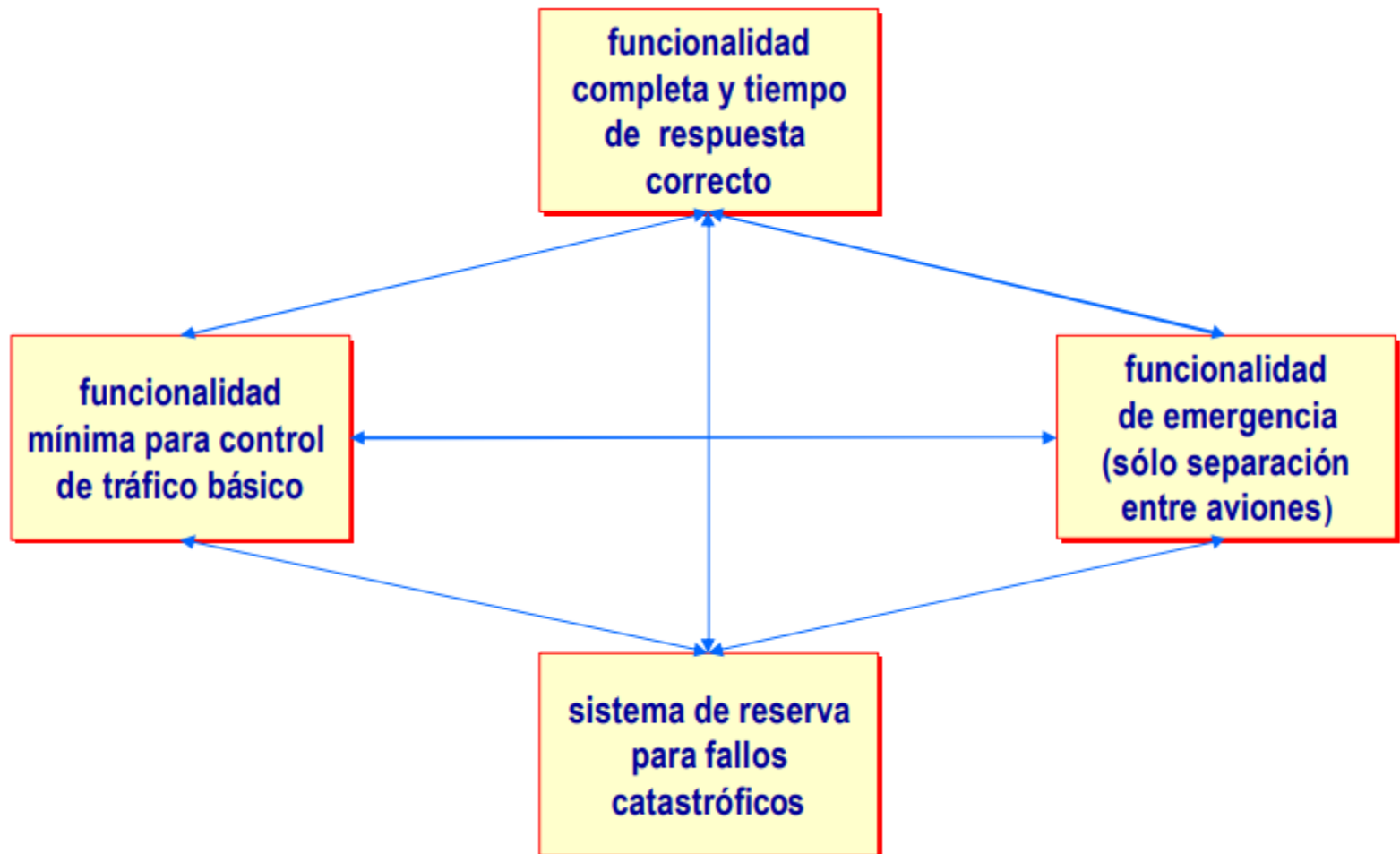
- **Los componentes de hardware fallan, a pesar de las técnicas de prevención**
 - La prevención es insuficiente si
 - la frecuencia o la duración de las reparaciones es inaceptable
 - no se puede detener el sistema para efectuar operaciones de mantenimiento
- **Ejemplo: naves espaciales no tripuladas**
- **La alternativa es utilizar técnicas de tolerancia de fallos**

Grados de tolerancia de fallos

- **Tolerancia completa (fail operational)**
 - El sistema sigue funcionando, al menos durante un tiempo, sin perder funcionalidad ni prestaciones
- **Degradación aceptable (fail soft, graceful degradation)**
 - El sistema sigue funcionando con una pérdida parcial de funcionalidad o prestaciones hasta la reparación del fallo
- **Parada segura (fail safe)**
 - El sistema se detiene en un estado que asegura la integridad del entorno hasta que se repare el fallo

El grado de tolerancia de fallos necesario depende de la aplicación

Ejemplo : control de tráfico aéreo



- **La tolerancia de fallos se basa en la redundancia**
- **Se utilizan componentes adicionales para detectar los fallos y recuperar el comportamiento correcto**
- **Esto aumenta la complejidad del sistema y puede introducir fallos adicionales**
- **Es mejor separar los componentes tolerantes del resto del sistema**



Redundancia en hardware

- **Redundancia estática**

- Los componentes redundantes están siempre activos
- Se utilizan para enmascarar los fallos
- Ejemplo:
 - Redundancia modular triple (ó N), TMR/NMR

- **Redundancia dinámica**

- Los componentes redundantes se activan cuando se detecta un fallo
- Se basa en la detección y posterior recuperación de los fallos
- Ejemplos:
 - sumas de comprobación » bits de paridad



Tolerancia de fallos de software

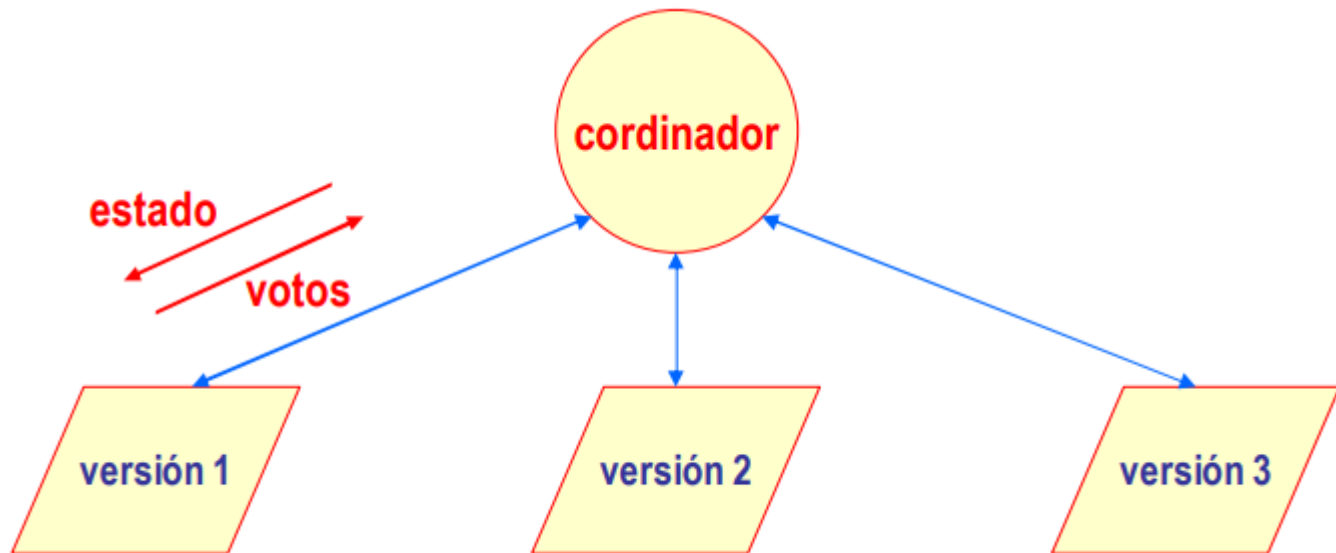
- **Técnicas para detectar y corregir errores de diseño**
- **Redundancia estática**
 - Programación con N versiones
- **Redundancia dinámica**
 - Dos etapas: detección y recuperación de fallos
 - Bloques de recuperación
 - Proporcionan recuperación hacia atrás
 - Excepciones
 - Proporcionan recuperación hacia adelante



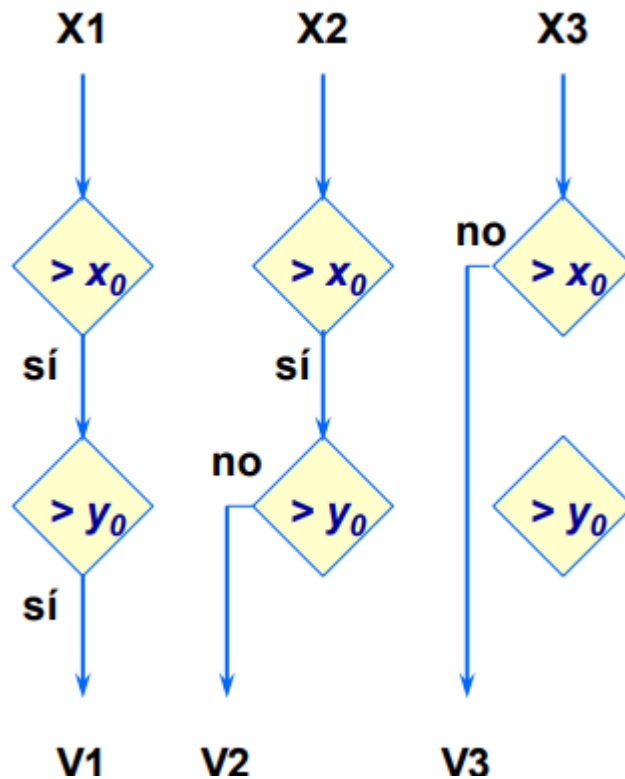
Programación con N versiones

- **Diversidad de diseño**
 - N programas desarrollados independientemente con la misma especificación
 - sin interacciones entre los equipos de desarrollo
- **Ejecución concurrente**
 - proceso coordinador (driver)
 - » intercambia datos con los procesos que ejecutan las versiones
 - todos los programas tienen las mismas entradas
 - las salidas se comparan
 - si hay discrepancia se realiza una votación

Programación con N versiones



Comparación consistente



- La comparación de valores reales o texto no es exacta
- Cada versión produce un resultado correcto, pero diferente de las otras
- No se arregla comparando con $x_0 + D$, $y_0 + D$

- **La correcta aplicación de este método depende de:**
 - Especificación inicial
 - Un error de especificación aparece en todas las versiones
 - Desarrollo independiente
 - No debe haber interacción entre los equipos
 - No está claro que distintos programadores cometan errores independientes
 - Presupuesto suficiente
 - Los costes de desarrollo se multiplican u ¿sería mejor emplearlos en mejorar una versión única?
 - El mantenimiento es también más costoso
- **Se ha utilizado en sistemas de aviónica críticos**

Redundancia dinámica en software

- **Cuatro etapas:**

1. Detección de errores

no se puede hacer nada hasta que se detecta un error

2. Evaluación y confinamiento de los daños »

diagnosis: averiguar hasta dónde ha llegado la información errónea

3. Recuperación de errores

llevar el sistema a un estado correcto, desde el que pueda seguir funcionando (tal vez con funcionalidad parcial)

4. Reparación de fallos

Aunque el sistema funcione, el fallo puede persistir y hay que repararlo